| BTS SERVICES INFORMATIQUES AUX ORGANISATIONS | SESSION 2026 |
|---|---|

**ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle (recto)**

**Épreuve E6 - Conception et développement d'applications (option SLAM)**

| **DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE** | **N° réalisation : 1** |
|---|---|
| **Nom, prénom : Frachisse Jessy** | **N° candidat :** |
| **Épreuve ponctuelle** ☐ **Contrôle en cours de formation** ☐ | **Date :** ..... / ..... /..... |

**Organisation support de la réalisation professionnelle**
Antiqua Média - Société d'édition de contenus pédagogiques numériques

**Intitulé de la réalisation professionnelle**
Civilipédia - Développement d'un wiki collaboratif sur les grandes civilisations du monde

**Période de réalisation :** 17/01/25 - 18/04/25 **Lieu :** Clermont-Ferrand

**Modalité :** ☐ **Seul(e)** ☑ **En équipe**

**Compétences travaillées**

- ☑ Concevoir et développer une solution applicative
- ☐ Assurer la maintenance corrective ou évolutive d'une solution applicative
- ☑ Gérer les données

**Conditions de réalisation[1] (ressources fournies, résultats attendus)**

La société Antiqua Média souhaitait disposer d'une application web de type wiki permettant la création, la modification et la consultation collaborative d'articles sur les grandes civilisations du monde (Mayas, Vikings, Mongols, Incas, Rome antique, etc.).

L'application devait gérer trois niveaux d'accès utilisateurs : visiteur non connecté, utilisateur authentifié et administrateur. Elle devait être responsive, intégrer un éditeur de texte enrichi (WYSIWYG), conserver un historique des versions de chaque article, et être développée selon une architecture découplée (client séparé de l'API).

Résultats attendus :

- Une API REST sécurisée par JWT
- Un client web PHP consommant l'API
- Un script SQL de création et d'initialisation de la base de données
- Une documentation utilisateur au format PDF
- Le code source versionné sur GitHub

[1] En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Administration des systèmes et des réseaux » prévues dans le référentiel de certification du BTS SIO.

**Description des ressources documentaires, matérielles et logicielles utilisées[2]**

Logiciels et frameworks :
- PHP 8.4 : langage principal (API et client)
- Symfony 7 : framework backend (API REST)
- Doctrine ORM : gestion de la base de données
- LexikJWT Bundle : authentification par token JWT (RS256)
- MySQL 8 : système de gestion de base de données
- Bootstrap 5 : framework CSS responsive
- Quill.js : éditeur de texte WYSIWYG
- Font Awesome 6 : icônes

Environnement de développement :
- XAMPP (Windows) : serveur Apache et MySQL local
- PHP Built-in Server : serveur de développement pour l'API Symfony
- Visual Studio Code : éditeur de code
- GitHub : versionnement et collaboration
- phpMyAdmin : administration de la base de données
- Swagger : test des endpoints de l'API

Documentation :
- Documentation officielle Symfony (symfony.com/doc)
- Documentation Bootstrap 5 (getbootstrap.com)
- Documentation Quill.js (quilljs.com)

**Modalités d'accès aux productions[3] et à leur documentation[4]**

https://sio.jbdelasalle.com/~jfrachisse/Portfolio_E6/index.php

**BTS SERVICES INFORMATIQUES AUX ORGANISATIONS** **SESSION 2026**

**ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle**

**(verso, éventuellement pages suivantes)**

**Épreuve E6 - Conception et développement d'applications (option SLAM)**

[2] Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

[3] Conformément au référentiel du BTS SIO « *Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve.* ». Les éléments nécessaires peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

[4] Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation, par exemples schéma complet de réseau mis en place et configurations des services.

**Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs**

## CONTEXTE ET PRÉSENTATION DU PROJET

**Civilipédia est une application web collaborative de type wiki, développée pour la société Antiqua Média. Elle permet à une communauté d'utilisateurs de créer, modifier et consulter des articles encyclopédiques sur les grandes civilisations du monde.**

**Le projet a été réalisé selon une architecture découplée imposée par le client : un back-end sous forme d'API REST (Symfony) et un front-end client PHP natif, communicant exclusivment via des requêtes HTTP JSON.**

## ARCHITECTURE TECHNIQUE

**Le projet est structuré en deux sous-projets distincts :**

**- civilipedia-api (port 3000)**
**API REST développée avec Symfony 7. Elle expose des endpoints sécurisés par authentification JWT (RS256 via LexikJWT). Elle gère l'ensemble de la logique métier : articles, utilisateurs, historique des versions, bannissements, contact et upload de fichiers.**

**- civilipedia-client (port 8000)**
**Client web PHP natif. Chaque page effectue des appels cURL vers l'API via une classe ApiClient.php centralisée. Le routage est géré par un paramètre GET (?page=...) via un point d'entrée unique (index.php).**

**Les fichiers uploadés (images d'articles, avatars) sont stockés dans un répertoire privé hors webroot côté API, et servis via des endpoints dédiés pour garantir la sécurité des accès.**

## FONCTIONNALITÉS PAR PROFIL UTILISATEUR

**Visiteur non connecté :**
**- Consultation de tous les articles**
**- Recherche en temps réel par mot-clé (titre et contenu)**
**- Consultation de l'historique des modifications (date, heure, auteur)**
**- Inscription et connexion**

**Utilisateur connecté :**
**- Création d'articles avec éditeur WYSIWYG (Quill.js)**
**- Modification de tout article avec éditeur WYSIWYG**
**- Suppression de ses propres articles**
**- Gestion du profil : changement d'avatar et de mot de passe**

**Administrateur :**
**- Suppression de n'importe quel article**
**- Bannissement temporaire ou définitif d'un utilisateur**
**- Débannissement d'un utilisateur**
**- Accès au tableau de bord de gestion des utilisateurs**

## POINTS TECHNIQUES NOTABLES

**- Authentification JWT RS256 : les tokens sont signés avec une paire de clés RSA asymétriques, plus sécurisée que HMAC. La durée de vie du token est configurable (JWT_ACCESS_TTL).**

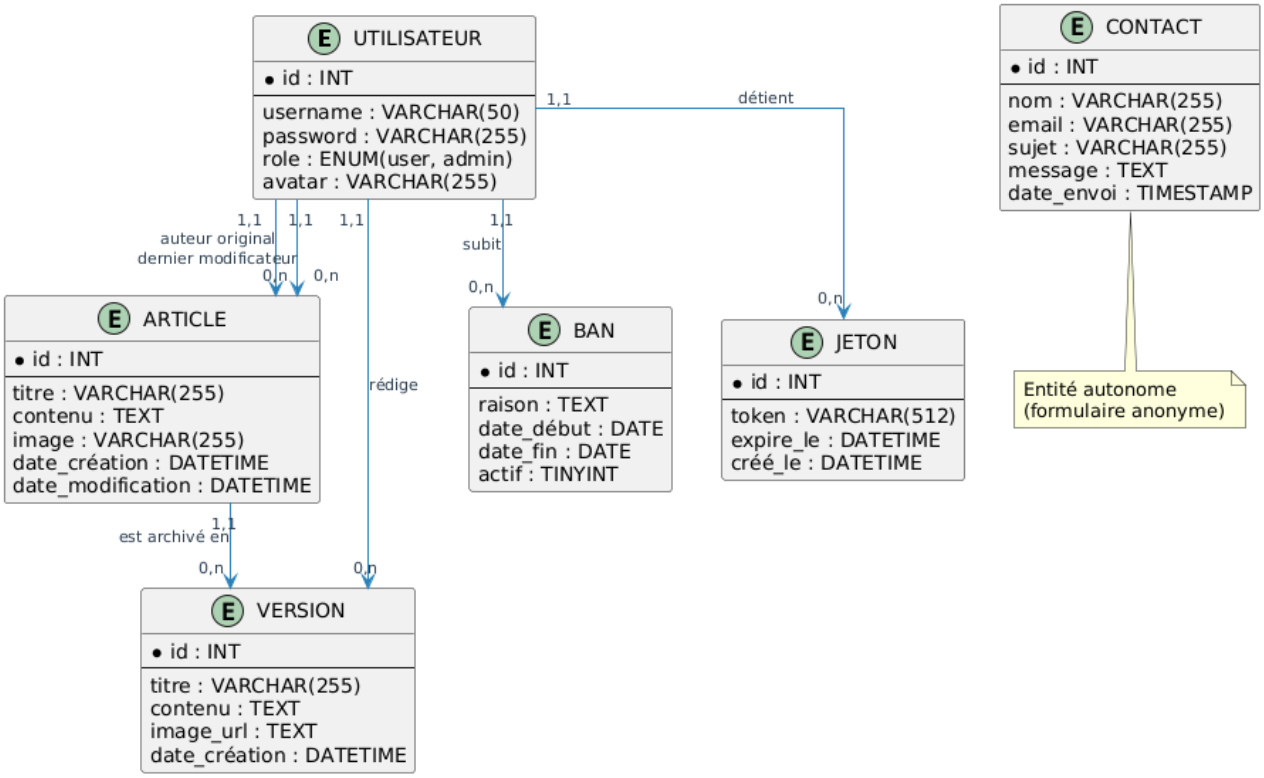
**- Upload sécurisé : les images (articles et avatars) sont stockées hors webroot dans /private/. L'accès se fait exclusivement via un endpoint API qui vérifie l'existence du fichier avant de le streamer.**

**- Cache-busting : les URLs d'images intègrent un paramètre ?t=timestamp pour forcer le rechargement après modification, contournant le cache navigateur.**

**- Éditeur WYSIWYG intégré : Quill.js est utilisé à la fois pour la création et la modification d'articles. Le contenu HTML produit est stocké tel quel en base et rendu côté client avec protection XSS.**

**- Recherche temps réel : la recherche filtre simultanément sur le titre et le contenu des articles côté client, sans rechargement de page, à partir des données déjà chargées depuis l'API.**

## ORGANISATION DU DÉVELOPPEMENT

**Le code source est versionné sur GitHub et partagé avec l'encadrant.**
**La base de données est initialisée via un fichier civilipedia.sql comprenant la création des tables et un jeu de données de test (17 articles, 4 utilisateurs, historique de versions).**

**Une documentation utilisateur complète (sans extrait de code, avec captures d'écran) est fournie au format PDF dans le dossier documentation/ du repository.**